

Papelaria SGC

Sistema de Gestao Comercial

ENTREGA 1

Modelagem e Arquitetura

Sistema de Gestao Comercial para a Papelaria Mundo Letrado

Disciplina:	Desenvolvimento Web
--------------------	---------------------

Projeto:	Sistema de Gestao Comercial (SGC) - Papelaria
-----------------	---

Equipe:	Pedro Henrique Bastos Guimaraes - 22303912
----------------	--

	Raphael Damiao Seabra Rosano - 22303595
--	---

Data:	Maio de 2026
--------------	--------------

Repositorio:	https://github.com/pedrobasgui/papelaria-sgc
---------------------	---

Sumario

1. Descricao do Sistema	3
2. Requisitos Funcionais	4
3. Requisitos Nao Funcionais	5
4. Arquitetura Proposta	6
5. Padroes de Projeto	8
6. Justificativa das Deciso es Tecnicas	9
7. Diagrama de Dominio	10
8. Diagrama de Classes	11
9. Diagrama Logico do Banco de Dados	12
10. Casos de Uso	13
11. Instrucoes de Execucao	16
12. Repositorio GitHub	17

1. Descricao do Sistema

A **Papeleria Mundo Letrado** e um pequeno comercio de bairro especializado em material escolar e de escritorio. Atualmente, todo o controle de vendas, estoque e cadastros e feito em planilhas e cadernos manuscritos, o que gera retrabalho, erros de contagem e dificulta a tomada de decisao do proprietario sobre reposicao de produtos e analise de faturamento.

O **Sistema de Gestao Comercial (SGC)** proposto neste projeto automatiza o ciclo completo da operacao da papelaria: cadastro de clientes e produtos, registro de vendas com baixa automatica de estoque, geracao de relatorios gerenciais e controle de acesso por perfis (Administrador e Funcionario). A solucao e composta por uma **API REST autenticada por JWT** e por uma **interface web construida em Django** que consome essa API.

O publico-alvo do sistema sao o(a) **proprietario(a)** da papelaria (perfil ADMIN, com acesso total ao sistema, incluindo cadastros e relatorios) e os **funcionarios do balcao** (perfil FUNCIONARIO, responsaveis pelo registro de vendas e consulta de catalogo).

Objetivos do Sistema

- Eliminar o controle manual de estoque, reduzindo perdas por divergencia.
- Centralizar o cadastro de clientes e historico de compras.
- Garantir baixa automatica do estoque a cada venda registrada.
- Oferecer relatorios visuais (graficos) de faturamento mensal e anual.
- Controlar quem pode fazer o que no sistema atraves de perfis.
- Permitir que o sistema seja acessado de qualquer dispositivo via web.

2. Requisitos Funcionais

Os requisitos funcionais descrevem **o que o sistema deve fazer**. Estao agrupados por modulo de negocio.

ID	Modulo	Descricao
RF-01	Autenticacao	Permitir login com username e senha; emitir token JWT valido.
RF-02	Autenticacao	Bloquear acesso a rotas protegidas sem token valido.
RF-03	Autenticacao	Restringir acoes administrativas ao perfil ADMIN.
RF-04	Cliente	Cadastrar, consultar, atualizar e excluir clientes.
RF-05	Cliente	Impedir CPF duplicado e validar formato do email.
RF-06	Cliente	Impedir exclusao de cliente que possua vendas registradas.
RF-07	Produto	Cadastrar, consultar, atualizar e excluir produtos.
RF-08	Produto	Impedir cadastro de produto com preco negativo.
RF-09	Produto	Sinalizar visualmente produtos abaixo do estoque minimo.
RF-10	Venda	Registrar venda com cliente, multiplos itens e usuario responsavel.
RF-11	Venda	Calcular automaticamente o valor total da venda.
RF-12	Venda	Bloquear venda quando estoque do produto for insuficiente.
RF-13	Venda	Atualizar (debitar) estoque de cada produto apos confirmacao da venda.
RF-14	Venda	Bloquear venda sem itens cadastrados.
RF-15	Relatorio	Gerar relatorio de vendas filtrado por periodo (data inicial / final).
RF-16	Relatorio	Gerar relatorio de vendas por cliente.
RF-17	Relatorio	Apresentar grafico de vendas anuais (mes a mes).
RF-18	Web	Disponibilizar tela de login, dashboard e CRUDs consumindo a API REST.

3. Requisitos Nao Funcionais

Os requisitos nao funcionais descrevem **como o sistema deve se comportar**: qualidade, restricoes e atributos. Cobrem seguranca, desempenho, usabilidade e manutenibilidade.

ID	Categoria	Descricao
RNF-01	Seguranca	Senhas armazenadas com hash PBKDF2 ou BCrypt - nunca em texto puro.
RNF-02	Seguranca	Autenticacao via JWT com expiracao do token de acesso em 60 minutos.
RNF-03	Seguranca	Token de refresh com validade maxima de 7 dias.
RNF-04	Seguranca	HTTPS obrigatorio em ambiente de producao.
RNF-05	Seguranca	Controle de acesso por perfil (ADMIN x FUNCIONARIO) em todos endpoints.
RNF-06	Desempenho	Tempo de resposta da API inferior a 500 ms em consultas simples.
RNF-07	Desempenho	Suportar pelo menos 50 usuarios simultaneos sem degradacao.
RNF-08	Usabilidade	Interface responsiva (Bootstrap 5) acessivel em desktop e mobile.
RNF-09	Usabilidade	Mensagens de erro claras em portugues, com codigo HTTP semantico.
RNF-10	Manutenibilidade	Codigo organizado em camadas (View -> Service -> Repository -> Model).
RNF-11	Manutenibilidade	Cobertura minima de 60% por testes automatizados.
RNF-12	Manutenibilidade	Padronizacao PEP-8 para Python e commits seguindo Conventional Commits.
RNF-13	Compatibilidade	Sistema executavel em Linux, Windows e macOS (Python 3.11+).
RNF-14	Persistencia	PostgreSQL em producao; SQLite para desenvolvimento e testes.
RNF-15	Versionamento	Codigo versionado em Git/GitHub com README e documentacao atualizados.
RNF-16	Documentacao	Endpoints documentados via Swagger/OpenAPI (drf-spectacular).

4. Arquitetura Proposta

O sistema adota uma **arquitetura em camadas (layered architecture)** que separa claramente as responsabilidades de apresentacao, regras de negocio e acesso a dados. Adicionalmente, ha uma separacao logica entre o **backend (API REST)** e o **frontend (interface web Django)**, que se comunicam exclusivamente via HTTP/JSON, garantindo que o frontend consuma a mesma API que poderia ser consumida por outros clientes (aplicativo mobile, integracoes, etc).

4.1 Camadas do Backend

Camada	Responsabilidade	Tecnologia
Apresentacao (Views)	Receber requisicoes HTTP, validar, retornar JSON.	DRF ViewSets / APIViews
Servico (Services)	Implementar regras de negocio e orquestrar operacoes.	Classes Python puras
Repositorio (Repositories)	Encapsular consultas e persistencia.	Django ORM via Manager
Modelo (Models)	Representar entidades de dominio e mapeamento ERD	Django Models
Persistencia (BD)	Armazenar e recuperar dados de forma duravel.	PostgreSQL / SQLite

4.2 Visao Geral do Fluxo

O fluxo tipico de uma requisicao no sistema:



4.3 Stack Tecnologica

- **Python 3.11+** - Linguagem principal.
- **Django 5** - Framework web e ORM.
- **Django REST Framework** - Construção da API REST e serialização.
- **djangorestframework-simplejwt** - Autenticação por token JWT.
- **PostgreSQL** - SGBD em produção (SQLite para desenvolvimento).
- **Bootstrap 5** - Componentes visuais e layout responsivo.
- **Chart.js** - Geração de gráficos no frontend para relatórios.

- **pytest-django** - Testes automatizados.
- **drf-spectacular** - Documentacao automatica em OpenAPI/Swagger.

5. Padroes de Projeto Utilizados

Foram aplicados padroes de projeto consagrados (GoF e padroes arquiteturais) para promover baixo acoplamento, alta coesao e facilidade de manutencao.

Repository (Estrutural / Acesso a Dados)

Encapsula a logica de consulta e persistencia, isolando o ORM. Cada agregado (Cliente, Produto, Venda) possui seu repositorio. Beneficia testes (mock facil) e troca de tecnologia de persistencia.

Service Layer (Arquitetural)

Regras de negocio centralizadas em classes de servico (ex.: VendaService.criar_venda) chamadas pelas Views. Evita 'fat models' e 'fat views', tornando a logica reutilizavel e testavel.

DTO (via Serializer) (Estrutural)

Os Serializers do DRF atuam como Data Transfer Objects, transformando entidades em JSON e vice-versa, validando entrada.

Strategy (Comportamental)

Aplicado no modulo de Relatorios: cada tipo de relatorio (por periodo, por cliente, anual) implementa uma estrategia distinta de calculo, intercambiavel via interface comum.

Singleton (Criacional)

Configuracao do Django (settings) e instancias de servicos sao carregadas uma unica vez por processo - padrao implicito do framework.

Decorator (Estrutural)

Uso intensivo de decoradores Python (@permission_classes, @action) para adicionar comportamento (autenticacao, autorizacao) sem alterar funcoes originais.

Observer (sinais) (Comportamental)

Sinais do Django (post_save) sao usados para reagir a eventos como criacao de venda, possibilitando expansao para auditoria/log.

6. Justificativa das Deciso es Tecnicas

Por que Django + Django REST Framework?

O enunciado exige **interface web em Django** e **API REST com JWT**. O Django ja oferece autenticao, ORM, sistema de templates e admin embutido. O DRF complementa com serializacao, viewsets e integracao trivial com JWT (via simplejwt). Combina-se assim a produtividade da plataforma 'batteries included' com a flexibilidade da API HTTP padronizada.

Por que PostgreSQL?

PostgreSQL e robusto, gratuito, possui excelente suporte a tipos numericos (NUMERIC para preco, evitando float), constraints (CHECK), indices avancados e e o SGBD mais bem suportado pelo Django. Em desenvolvimento usamos SQLite por simplicidade (zero configuracao).

Por que JWT em vez de sessao?

JWT e **stateless**: o servidor nao precisa armazenar sessao, o que facilita escalabilidade horizontal e desacopla o backend do frontend. Como o frontend Django consome a API como um cliente qualquer, o token tambem servira para um eventual app mobile.

Por que arquitetura em camadas?

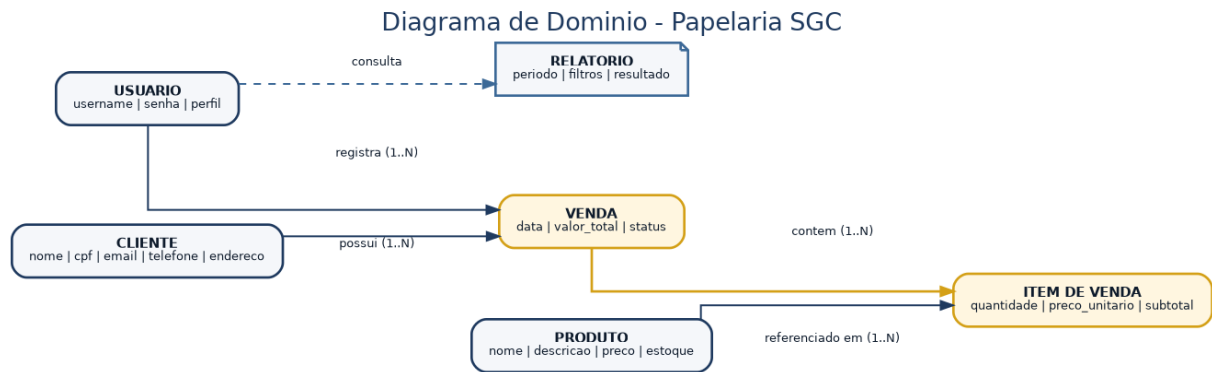
Separar Apresentacao, Servico, Repositorio e Modelo facilita **manutencao** (cada camada tem uma responsabilidade clara), **testabilidade** (mocks por interface) e **evolucao** (trocar DB ou framework web sem reescrever regras de negocio).

Por que Bootstrap 5 + Chart.js no frontend?

Bootstrap 5 entrega componentes prontos e responsivos sem necessidade de SPA, mantendo o stack simples. Chart.js e leve, tem boa documentacao e atende plenamente os relatorios graficos exigidos (grafico de vendas anuais).

7. Diagrama de Dominio

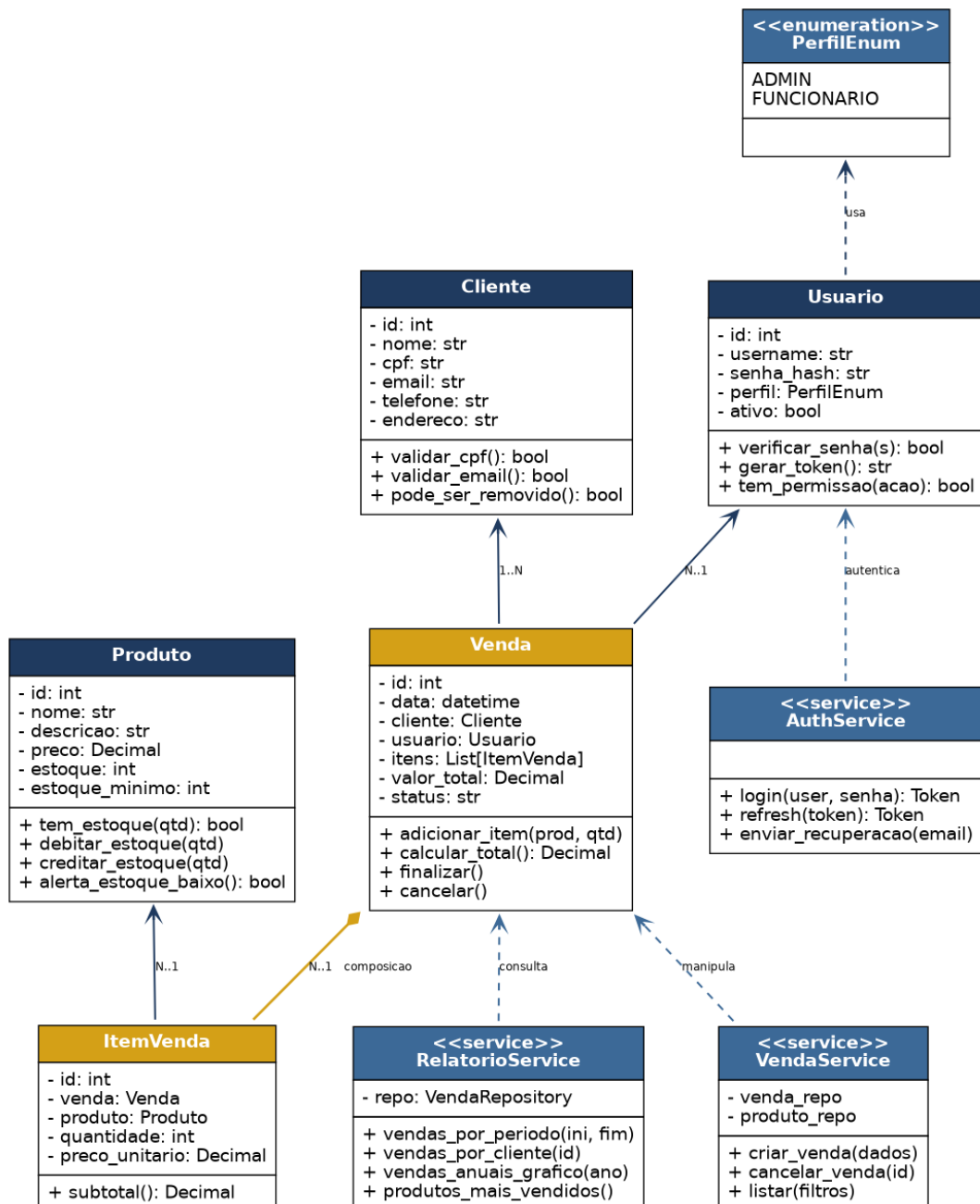
Representa as entidades centrais do negocio e seus relacionamentos. **Venda** e **Item de Venda** formam o agregado principal (destacados em dourado).



8. Diagrama de Classes

Refina o diagrama de dominio com atributos, metodos e visibilidade. Inclui as classes de **servico** (VendaService, RelatorioService, AuthService) e a enumeracao **PerfilEnum**. A composicao entre **Venda** e **ItemVenda** esta marcada (em dourado) - itens nao existem sem uma venda.

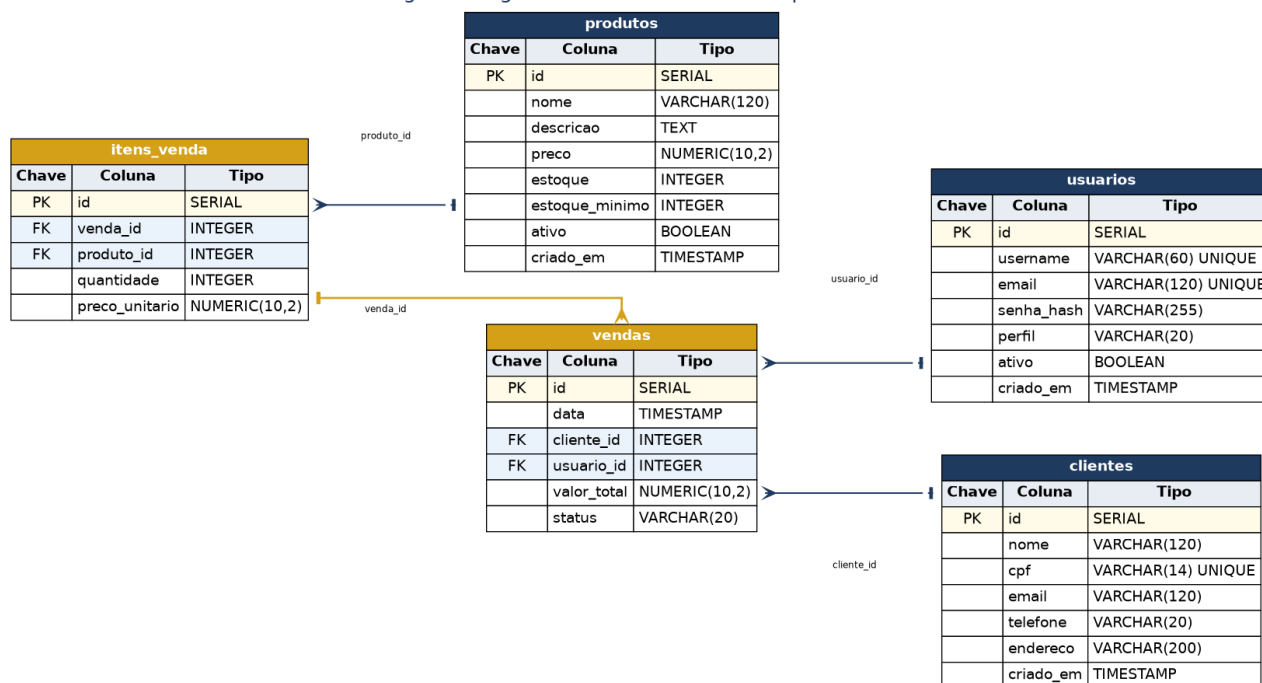
Diagrama de Classes - Papellaria SGC



9. Diagrama Logico do Banco de Dados

Modelo relacional normalizado (3FN). Chaves primarias destacadas em amarelo claro, chaves estrangeiras em azul claro. Foram criados indices em colunas de filtro frequente (cpf, nome, data, status) para garantir desempenho em consultas.

Diagrama Logico do Banco de Dados - Papellaria SGC



Regras de integridade implementadas no SQL:

- CPF unico em **clientes**.
- Username e email unicos em **usuarios**.
- **preco** e **estoque** com CHECK >= 0.
- **quantidade** de item de venda com CHECK > 0.
- ON DELETE RESTRICT em **cliente_id** e **usuario_id** de venda (impede exclusao de cliente/usuario com vendas).
- ON DELETE CASCADE em **itens_venda.venda_id** (cancelar venda apaga itens).
- Status da venda restrito a 'CONCLUIDA' ou 'CANCELADA'.

10. Casos de Uso

Os principais casos de uso do sistema são descritos a seguir, associados aos atores **Administrador** e **Funcionario**.

UC-01 - Realizar Login

Ator(es)	Administrador, Funcionario
Pre-condicao	Usuario possui credenciais validas no sistema.
Fluxo principal	1. Usuario acessa a tela de login. 2. Informa username e senha. 3. Sistema valida credenciais. 4. Sistema emite token JWT (acesso + refresh). 5. Usuario e redirecionado para o dashboard.
Fluxo alternativo	Credenciais invalidas: sistema exibe mensagem 'Usuario ou senha invalidos'.
Pos-condicao	Usuario autenticado com token armazenado no navegador.

UC-02 - Cadastrar Cliente

Ator(es)	Administrador, Funcionario
Pre-condicao	Usuario autenticado no sistema.
Fluxo principal	1. Usuario acessa o menu 'Clientes' > 'Novo'. 2. Preenche nome, CPF, email, telefone e endereco. 3. Sistema valida CPF (formato e unicidade) e email. 4. Sistema persiste o cliente no banco. 5. Sistema retorna mensagem de sucesso e redireciona para a listagem.
Fluxo alternativo	CPF ja cadastrado: sistema exibe 'CPF ja existe'. Email invalido: sistema exibe 'Formato de email invalido'.
Pos-condicao	Cliente disponivel para selecao em vendas futuras.

UC-03 - Cadastrar Produto

Ator(es)	Administrador
Pre-condicao	Usuario autenticado com perfil ADMIN.

Fluxo principal	1. Acessa o menu 'Produtos' > 'Novo'. 2. Informa nome, descricao, preco, estoque e estoque minimo. 3. Sistema valida preco ≥ 0 e estoque ≥ 0 . 4. Sistema persiste o produto.
Fluxo alternativo	Preco negativo: sistema exibe erro de validacao.
Pos-condicao	Produto disponivel no catalogo.

UC-04 - Registrar Venda

Ator(es)	Administrador, Funcionario
Pre-condicao	Usuario autenticado; cliente ja cadastrado; produtos com estoque.
Fluxo principal	1. Usuario acessa 'Vendas' > 'Nova'. 2. Seleciona cliente. 3. Adiciona itens (produto + quantidade). 4. Sistema verifica estoque disponivel para cada item. 5. Sistema calcula valor total automaticamente. 6. Usuario confirma a venda. 7. Sistema persiste a venda, debita o estoque dos produtos e emite recibo.
Fluxo alternativo	Estoque insuficiente: sistema bloqueia inclusao do item. Venda sem itens: botao 'Confirmar' permanece desabilitado.
Pos-condicao	Venda registrada e estoque atualizado.

UC-05 - Consultar Relatorio de Vendas por Periodo

Ator(es)	Administrador
Pre-condicao	Usuario autenticado com perfil ADMIN.
Fluxo principal	1. Acessa 'Relatorios' > 'Por periodo'. 2. Informa data inicial e final. 3. Sistema retorna lista de vendas e valor total no periodo.
Fluxo alternativo	Periodo invalido (data final < data inicial): sistema exibe erro.
Pos-condicao	Relatorio exibido em tabela com opcao de download.

UC-06 - Visualizar Grafico de Vendas Anuais

Ator(es)	Administrador
-----------------	---------------

Pre-condicao	Usuario autenticado com perfil ADMIN.
Fluxo principal	1. Acessa 'Relatorios' > 'Anual'. 2. Seleciona o ano desejado. 3. Sistema retorna serie agregada por mes. 4. Frontend renderiza grafico de barras (Chart.js).
Fluxo alternativo	Sem vendas no ano: grafico exibido vazio com aviso.
Pos-condicao	Grafico apresentado para analise gerencial.

UC-07 - Excluir Cliente

Ator(es)	Administrador
Pre-condicao	Cliente existente sem vendas associadas.
Fluxo principal	1. Acessa listagem de clientes. 2. Clica em 'Excluir' no cliente desejado. 3. Confirma a exclusao. 4. Sistema verifica se o cliente possui vendas. 5. Sistema remove o cliente.
Fluxo alternativo	Cliente possui vendas: sistema bloqueia e exibe mensagem.
Pos-condicao	Cliente removido da base.

11. Instrucoes para Executar o Sistema

Pre-requisitos

- Python 3.11 ou superior
- PostgreSQL 13+ (opcional - SQLite e usado por padrao em desenvolvimento)
- Git
- Node nao e necessario - frontend usa CDN do Bootstrap e Chart.js

Passos para subir o ambiente

```
# 1. Clonar o repositario
git clone https://github.com/pedrobasgui/papelaria-sgc.git
cd papelaria-sgc

# 2. Criar e ativar ambiente virtual
python -m venv .venv
source .venv/bin/activate      # Linux/Mac
# .venv\Scripts\activate      # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Aplicar migracoes (cria as tabelas)
python manage.py migrate

# 5. Carregar dados iniciais (opcional)
python manage.py loaddata fixtures/seed.json

# 6. Criar superusuario
python manage.py createsuperuser

# 7. Rodar o servidor
python manage.py runserver
```

Apos o passo 7, o sistema fica disponivel em **<http://localhost:8000>** e a documentacao da API em **<http://localhost:8000/api/docs/>**.

12. Repositorio GitHub

O código-fonte completo, os scripts SQL, este documento e os diagramas estão versionados no repositório público:

<https://github.com/pedrobasgui/papelaria-sgc>

Estrutura do repositório

```
papelaria-sgc/  
|-- README.md  
|-- requirements.txt  
|-- manage.py  
|-- .gitignore  
|-- docs/  
|   |-- Entrega1_Documentacao.pdf  
|   |-- diagrama_dominio.png  
|   |-- diagrama_classes.png  
|   |-- diagrama_banco.png  
|   |-- casos_de_uso.md  
|-- database/  
|   |-- script.sql  
|-- papelaria_sgc/      (configuracoes Django)  
|-- apps/  
|   |-- core/           (execucoes, permissoes, handlers)  
|   |-- usuarios/       (auth, JWT, perfil)  
|   |-- clientes/        (CRUD)  
|   |-- produtos/        (CRUD + estoque)  
|   |-- vendas/          (vendas + itens)  
|   |-- relatorios/      (estrategias de relatorio)  
|   |-- web/             (views Django + templates)  
|-- templates/          (HTML)  
|-- static/             (CSS / JS / imagens)
```

Fim do documento - Entrega 1